

1) ATENÇÃO: aqui temos uma explicação completa do problema. A sua resposta poderia ser resumida nas partes destacadas em verde escuro.

Este projeto, apesar de usar técnicas amplamente conhecidas da programação orientada por objetos como a herança e a agregação, desobedece quase completamente os princípios de modularidade e SOLID. Não era obrigatório responder usando SOLID, mas, como já mencionei com a turma, usar os princípios é sempre um facilitador, pois são conceitos bem estabelecidos e aceitos. Ao demonstrar que um projeto é aderente ao SOLID, ou que contradiz o SOLID, fica fácil perceber se ele é aceitável ou tem muitos problemas. Vou, então, usar as letras para exemplificar os problemas:

S – Apesar de as classes aparentemente terem responsabilidades únicas bem definidas, podemos criticar o fato de, por exemplo, não haver métodos separados para os cálculos das despesas com combustível e com impostos.

O – Da maneira como está, o princípio do aberto/fechado está sendo desrespeitado em diversos momentos, apesar da existência de herança. Cada tipo de *Veículo* está usando um nome de método diferente para sua despesa, obrigando que a *Frota* seja modificada sempre que aparecer um novo tipo de *Veículo*. A própria existência de um atributo “tipoVeiculo” na classe *Veiculo* indica um desrespeito ao OCP, induzindo uma necessidade de verificar este atributo para fazer uma chamada de método que poderia ser polimórfica.

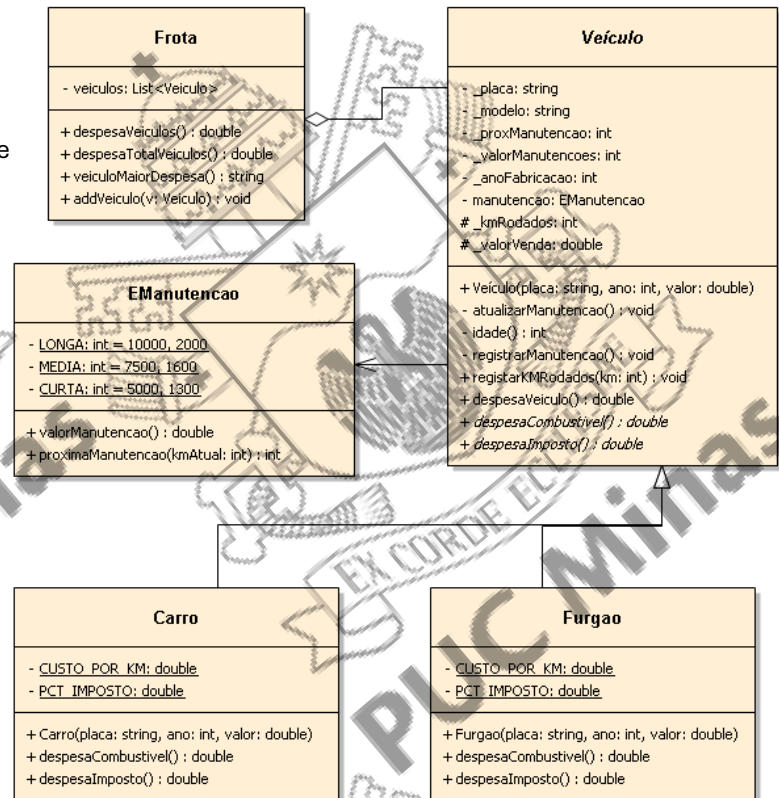
L – Como consequência do desrespeito acima, corremos um enorme risco de desrespeitar o LSP na classe *Frota*. Se temos um veículo na lista “veículos” desta *Frota* e substituímos por uma classe filha ou outra, não é garantido que o método de despesa será chamado corretamente. A classe mãe não está podendo ser substituída de maneira correta e consistente por não haver um polimorfismo efetivo na herança. Para “contornar o problema”, foram escritos vários métodos duplicados na classe *Frota* para adicionar veículos e calcular suas despesas.

Ainda podemos citar o fato de que os atributos “custoPorKM” e “pctImposto” poderiam ser constantes nas classes filhas de *Veiculo*, o fato de que atributos como “placa” e “modelo” aparentemente não precisarem ser protegidos, podendo ser privados, e o fato de que *Frota* tem vários métodos duplicados com lógica igual por causa da herança mal feita.

A partir das observações acima, a solução mais direta seria modificar a maneira como a herança está feita para se adequar aos princípios OCP e LSP:

- Transformar a classe *Veiculo* em abstrata, com um métodos abstratos “despesaCombustivel” e “despesaImposto”. Esta classe teria um método “despesaVeiculo” que faria a chamada somando estes dois métodos abstratos.
- Transformar os atributos “custoPorKM” e “pctImposto” das classes filhas em constantes. Estas constantes seriam usadas nas classes filhas para implementar os métodos abstratos citados acima.
- Os atributos protegidos precisariam ser apenas “valorVenda” e “kmRodados”. Os demais não têm utilidade para as classes filhas.
- Assim, adequaríamos a classe *Frota*, que poderia substituir todos os métodos duplicados por métodos mais genéricos: “despesaTotalVeiculos”, “addVeiculo” no lugar de um método para cada tipo de *Veiculo*.

2) Como se trata de uma questão de modelagem, várias soluções podem ser feitas e admitidas como corretas. O importante é respeitar a modularidade (coesão, baixo acoplamento, abstração, encapsulamento) e os princípios SOLID. Esta solução usa herança para os tipos de Veículo e enumerador para as manutenções, pois estas têm valores constantes (próxima manutenção e valor). O veículo possui um objeto de EManutencao e, no seu construtor, este objeto seria criado e atribuído com "LONGA". O atributo "proxManutencao" seria atualizado com o valor retornado por este enumerador. Sempre que houver um registro de quilômetros rodados, será chamado o método registrarManutencao, que fará a coordenação das ações chamando os métodos de registrar manutenção e atualizar manutenção, se for o caso.



3) O cálculo da quilometragem da próxima manutenção é simples, sendo igual para qualquer EManutencao:

```

CLASSE Veiculo:
private double registrarManutencao(){
    valorManutencao += manutencao.valorManutencao();
    atualizarManutencao();
    proxManutencao = manutencao.proximaManutencao(kmRodados);
    return valorManutencao;
}

private void atualizarManutencao(){
    manutencao = EManutencao.CURTA;
    if(idade() < 3)
        manutencao = EManutencao.LONGA;
    else if(idade() < 6)
        manutencao = EManutencao.MEDIA;
}
  
```

Não foi cobrado na questão, mas coloco aqui como ilustração método público de registrar km que dispara a ação:

```

public int registrarKMRodados(int km){
    if(km < 0)
        throw new IllegalArgumentException("Quilometragem inválida");
    kmRodados += km;
    if(kmRodados >= proxManutencao)
        registrarManutencao();
    return kmRodados;
}
  
```

4)CLASSE Frota:

```

public string veiculoMaiorDespesa(){
    Veiculo maiorDespesa = veiculos.get(0);
    double valorDespesa = maiorDespesa.despesaVeiculo();
    for(Veiculo veiculo : veiculos){
        if(veiculo.despesaVeiculo() > valorDespesa){
            maiorDespesa = veiculo;
            valorDespesa = maiorDespesa.despesaVeiculo();
        }
    }
    return maiorDespesa.toString();
}
  
```

5)
CLASSE Veiculo:

```
public double despesaVeiculo(){  
    return despesaCombustivel() + despesaImposto() + _valorManutencoes;  
}
```